Regresión Multivariada

Selección de variables predictoras — Actividad Final: Real Estate

J. Emiliano Hdz. Montalvo A00574526

In [1]:
```
#  Importar bibliotecas necesarias
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn import metrics
from sklearn.feature_selection import f_regression, SelectKBest

import statsmodels.api as sm
from statsmodels.stats.outliers_influence import variance_inflation_factor

%matplotlib inline
```

Vamos a importar la base de datos real para la actividad:

In [2]:
```
#  Importamos el problema de bienes raíces (Real Estate)
data_raw = pd.read_csv("Realestate.csv")

# Renombramos las columnas para facilitar el análisis
df = data_raw.rename(columns={
    'X1 transaction date': 'Fecha_transaccion',
    'X2 house age': 'Antigüedad',
    'X3 distance to the nearest MRT station': 'Distancia_estacion',
    'X4 number of convenience stores': 'Tiendas_conveniencia',
    'X5 latitude': 'Latitud',
    'X6 longitude': 'Longitud',
    'Y house price of unit area': 'Precio'
})

# Eliminamos la columna de índice que no aporta información predictiva
df = df.drop(columns=['No'])

print(f"Forma del dataset: {df.shape}")
print("\nPrimeras 5 filas:")
df.head()
```

```
Forma del dataset: (414, 7)

Primeras 5 filas:
```

Out[2]:

	Fecha_transaccion	Antigüedad	Distancia_estacion	Tiendas_conveniencia	
0	2012.917	32.0	84.87882	10	
1	2012.917	19.5	306.59470	9	
2	2013.583	13.3	561.98450	5	
3	2013.500	13.3	561.98450	5	
4	2012.833	5.0	390.56840	5	

In [3]:
```
# Estadísticas descriptivas
print("ESTADÍSTICAS DESCRIPTIVAS\n")
print(df.describe().round(2))
```

```
ESTADÍSTICAS DESCRIPTIVAS

       Fecha_transaccion  Antigüedad  Distancia_estacion  \
count             414.00      414.00              414.00
mean             2013.15       17.71             1083.89
std                 0.28       11.39             1262.11
min              2012.67        0.00               23.38
25%              2012.92        9.02              289.32
50%              2013.17       16.10              492.23
75%              2013.42       28.15             1454.28
max              2013.58       43.80             6488.02

       Tiendas_conveniencia  Latitud  Longitud  Precio
count                414.00   414.00    414.00  414.00
mean                   4.09    24.97    121.53   37.98
std                    2.95     0.01      0.02   13.61
min                    0.00    24.93    121.47    7.60
25%                    1.00    24.96    121.53   27.70
50%                    4.00    24.97    121.54   38.45
75%                    6.00    24.98    121.54   46.60
max                   10.00    25.01    121.57  117.50
```

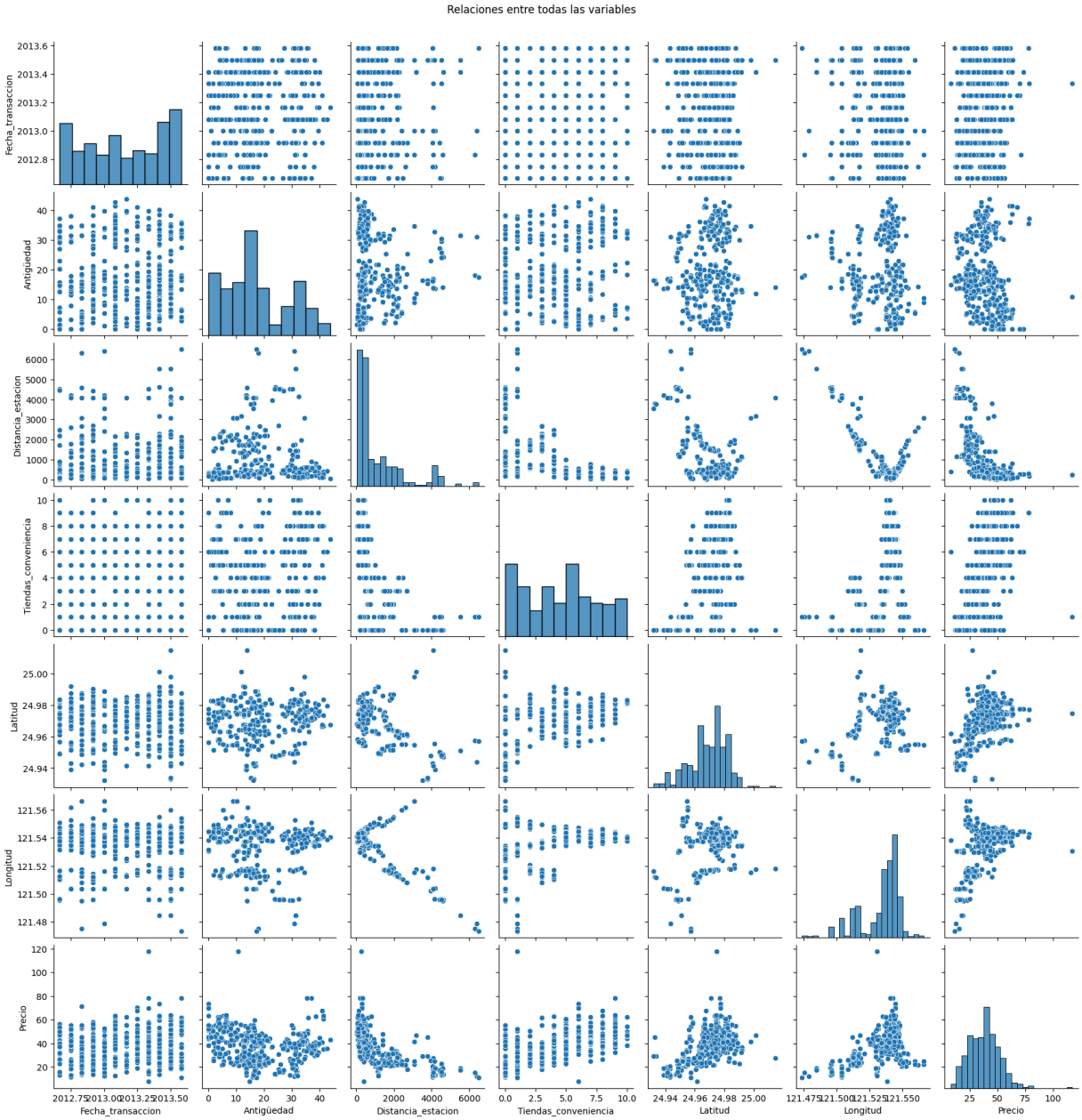
In [4]:
```
# Matriz de correlación detallada
print("\nMATRIZ DE CORRELACIÓN\n")
print(df.corr().round(3))
```

```
MATRIZ DE CORRELACIÓN

                     Fecha_transaccion  Antigüedad  Distancia_estacion  \
Fecha_transaccion                1.000       0.018               0.061
Antigüedad                       0.018       1.000               0.026
Distancia_estacion               0.061       0.026               1.000
Tiendas_conveniencia             0.010       0.050              -0.603
Latitud                          0.035       0.054              -0.591
Longitud                        -0.041      -0.049              -0.806
Precio                           0.087      -0.211              -0.674

                     Tiendas_conveniencia  Latitud  Longitud  Precio
Fecha_transaccion                   0.010    0.035    -0.041   0.087
Antigüedad                          0.050    0.054    -0.049  -0.211
Distancia_estacion                 -0.603   -0.591    -0.806  -0.674
Tiendas_conveniencia                1.000    0.444     0.449   0.571
Latitud                             0.444    1.000     0.413   0.546
Longitud                            0.449    0.413     1.000   0.523
Precio                              0.571    0.546     0.523   1.000
```

In [5]:
```python
# Pairplot de variables
sns.pairplot(df)
plt.suptitle('Relaciones entre todas las variables', y=1.02)
plt.show()
```



In [6]:
```
#  Dividir datos en entrenamiento y prueba
X = df.drop('Precio', axis=1)
y = df['Precio']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

print(f"Tamaño de entrenamiento: {X_train.shape[0]} muestras")
print(f"Tamaño de prueba: {X_test.shape[0]} muestras")
```

```
Tamaño de entrenamiento: 289 muestras
Tamaño de prueba: 125 muestras
```

In [7]:
```
# Entrenar modelo de regresión lineal con todas las variables
modelo = LinearRegression()
modelo.fit(X_train, y_train)
```

```
# Mostrar coeficientes
coeficientes = pd.DataFrame({
    'Variable': X.columns,
    'Coeficiente': modelo.coef_
})
print("=== COEFICIENTES DEL MODELO ===\n")
print(coeficientes)
print(f"\nIntercept (término constante): {modelo.intercept_:.4f}")
```

```
=== COEFICIENTES DEL MODELO ===

               Variable  Coeficiente
0     Fecha_transaccion     5.847797
1             Antigüedad    -0.242546
2    Distancia_estacion    -0.005139
3  Tiendas_conveniencia     1.074495
4               Latitud   239.096949
5              Longitud   -52.235159

Intercept (término constante): -11350.7405
```

In [8]:
```
# Evaluar modelo completo
y_pred = modelo.predict(X_test)

MAE = metrics.mean_absolute_error(y_test, y_pred)
MSE = metrics.mean_squared_error(y_test, y_pred)
RMSE = np.sqrt(MSE)
R2 = metrics.r2_score(y_test, y_pred)

print("=== MÉTRICAS DE EVALUACIÓN ===\n")
print(f"Error Absoluto Medio (MAE): {MAE:.4f}")
print(f"Error Cuadrático Medio (MSE): {MSE:.4f}")
print(f"Raíz del Error Cuadrático Medio (RMSE): {RMSE:.4f}")
print(f"R² (Coeficiente de Determinación): {R2:.4f}")
```

```
=== MÉTRICAS DE EVALUACIÓN ===

Error Absoluto Medio (MAE): 6.1848
Error Cuadrático Medio (MSE): 73.5684
Raíz del Error Cuadrático Medio (RMSE): 8.5772
R² (Coeficiente de Determinación): 0.5601
```

In [9]:
```
# Análisis de residuos
residuos = y_test - y_pred

plt.figure(figsize=(12, 4))

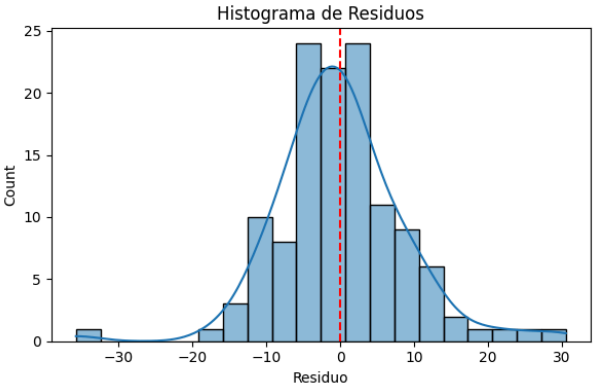
plt.subplot(1, 2, 1)
sns.histplot(residuos, kde=True, bins=20)
plt.axvline(x=0, color='r', linestyle='--')
plt.title('Histograma de Residuos')
plt.xlabel('Residuo')

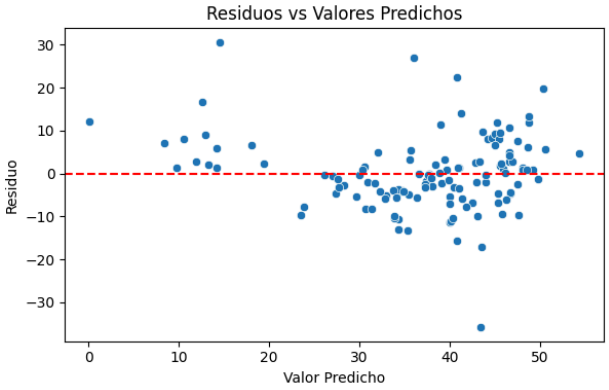
plt.subplot(1, 2, 2)
sns.scatterplot(x=y_pred, y=residuos)
plt.axhline(y=0, color='r', linestyle='--')
plt.title('Residuos vs Valores Predichos')
```

```
plt.xlabel('Valor Predicho')
plt.ylabel('Residuo')

plt.tight_layout()
plt.show()

print(f"Media de residuos: {residuos.mean():.4f}")
print(f"Desviación estándar de residuos: {residuos.std():.4f}")
```






```
Media de residuos: -0.0122
Desviación estándar de residuos: 8.6117
```

In [10]:
```
# Cálculo del VIF (Factor de Inflación de la Varianza)
X_constant = sm.add_constant(X)
vif_data = pd.DataFrame()
vif_data["Variable"] = X_constant.columns
vif_data["VIF"] = [variance_inflation_factor(X_constant.values, i)
                   for i in range(X_constant.shape[1])]

print("=== FACTOR DE INFLACIÓN DE LA VARIANZA (VIF) ===\n")
print(vif_data)
print("\n Recuerda:")
print("- VIF > 5: indica alta multicolinealidad")
print("- VIF > 10: indica multicolinealidad severa")
```

```
=== FACTOR DE INFLACIÓN DE LA VARIANZA (VIF) ===

               Variable       VIF
0                 const  1.000000
1     Fecha_transaccion  1.014674
2             Antigüedad  1.014287
3    Distancia_estacion  4.323019
4  Tiendas_conveniencia  1.617038
5               Latitud  1.610234
6              Longitud  2.926302

 Recuerda:
- VIF > 5: indica alta multicolinealidad
- VIF > 10: indica multicolinealidad severa
```

In [11]:
```
#  Selección hacia adelante (Forward Selection)
def forward_selection(X, y, threshold=0.05):
    """
    Selección de variables hacia adelante basada en p-valor
    """
```

```
    included = []
    while True:
        changed = False
        excluded = list(set(X.columns) - set(included))
        new_pval = pd.Series(index=excluded, dtype=float)

        for new_col in excluded:
            model = sm.OLS(y, sm.add_constant(pd.DataFrame(X[included + [new
            new_pval[new_col] = model.pvalues[new_col]

        best_pval = new_pval.min()
        if best_pval < threshold:
            best_feature = new_pval.idxmin()
            included.append(best_feature)
            changed = True

        if not changed:
            break

    return included

print("=== SELECCIÓN HACIA ADELANTE ===\n")
variables_seleccionadas = forward_selection(X, y)
print(f"Variables seleccionadas: {variables_seleccionadas}")
```

```
=== SELECCIÓN HACIA ADELANTE ===
Variables seleccionadas: ['Distancia_estacion', 'Tiendas_conveniencia', 'Ant
igüedad', 'Latitud', 'Fecha_transaccion']
```

In [12]:

```
# Comparación de modelos con diferentes variables
print("=== COMPARACIÓN DE MODELOS ===\n")

combinaciones = [
    ['Distancia_estacion'],
    ['Distancia_estacion', 'Tiendas_conveniencia'],
    ['Distancia_estacion', 'Tiendas_conveniencia', 'Antigüedad'],
    ['Distancia_estacion', 'Tiendas_conveniencia', 'Antigüedad', 'Latitud'],
    ['Distancia_estacion', 'Tiendas_conveniencia', 'Antigüedad', 'Latitud',
    variables_seleccionadas
]

nombres_combinaciones = [
    'Modelo 1: Distancia_estacion',
    'Modelo 2: Distancia_estacion + Tiendas_conveniencia',
    'Modelo 3: Distancia_estacion + Tiendas_conveniencia + Antigüedad',
    'Modelo 4: Distancia_estacion + Tiendas_conveniencia + Antigüedad + Lati
    'Modelo 5: Modelo completo',
    'Modelo 6: Variables seleccionadas (Forward)'
]

for nombre, vars_list in zip(nombres_combinaciones, combinaciones):
    X_sel = sm.add_constant(X[vars_list])
    model = sm.OLS(y, X_sel).fit()

    print(f"📊 {nombre}")
    print(f"  Variables: {vars_list}")
```

```
    print(f"  R² = {model.rsquared:.4f}")
    print(f"  R² ajustado = {model.rsquared_adj:.4f}")
    print(f"  AIC = {model.aic:.2f}")
    print(f"  BIC = {model.bic:.2f}")
    print()
```

```
=== COMPARACIÓN DE MODELOS ===

📊 Modelo 1: Distancia_estacion
  Variables: ['Distancia_estacion']
  R² = 0.4538
  R² ajustado = 0.4524
  AIC = 3089.07
  BIC = 3097.12

📊 Modelo 2: Distancia_estacion + Tiendas_conveniencia
  Variables: ['Distancia_estacion', 'Tiendas_conveniencia']
  R² = 0.4966
  R² ajustado = 0.4941
  AIC = 3057.28
  BIC = 3069.36

📊 Modelo 3: Distancia_estacion + Tiendas_conveniencia + Antigüedad
  Variables: ['Distancia_estacion', 'Tiendas_conveniencia', 'Antigüedad']
  R² = 0.5411
  R² ajustado = 0.5377
  AIC = 3020.97
  BIC = 3037.08

📊 Modelo 4: Distancia_estacion + Tiendas_conveniencia + Antigüedad + Latitu
d
  Variables: ['Distancia_estacion', 'Tiendas_conveniencia', 'Antigüedad', 'L
atitud']
  R² = 0.5711
  R² ajustado = 0.5669
  AIC = 2994.91
  BIC = 3015.04

📊 Modelo 5: Modelo completo
  Variables: ['Distancia_estacion', 'Tiendas_conveniencia', 'Antigüedad', 'L
atitud', 'Longitud', 'Fecha_transaccion']
  R² = 0.5824
  R² ajustado = 0.5762
  AIC = 2987.92
  BIC = 3016.11

📊 Modelo 6: Variables seleccionadas (Forward)
  Variables: ['Distancia_estacion', 'Tiendas_conveniencia', 'Antigüedad', 'L
atitud', 'Fecha_transaccion']
  R² = 0.5823
  R² ajustado = 0.5772
  AIC = 2985.99
  BIC = 3010.15
```

In [13]:

```
# Análisis detallado del mejor modelo
print("=== MEJOR MODELO SEGÚN FORWARD SELECTION ===\n")

X_sel = sm.add_constant(X[variables_seleccionadas])
modelo_final = sm.OLS(y, X_sel).fit()
print(modelo_final.summary())
```

```
=== MEJOR MODELO SEGÚN FORWARD SELECTION ===

                            OLS Regression Results
==============================================================================
Dep. Variable:                 Precio   R-squared:                       0.582
Model:                            OLS   Adj. R-squared:                  0.577
Method:                 Least Squares   F-statistic:                     113.8
Date:                Thu, 17 Sep 2026   Prob (F-statistic):           4.47e-75
Time:                        22:01:59   Log-Likelihood:                -1487.0
No. Observations:                 414   AIC:                             2986.
Df Residuals:                     408   BIC:                             3010.
Df Model:                           5
Covariance Type:            nonrobust
==============================================================================
                             coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const                  -1.596e+04   3233.450     -4.936      0.000   -2.23e+04   -9602.961
Distancia_estacion      -0.0044      0.000     -8.887      0.000      -0.005      -0.003
Tiendas_conveniencia     1.1361      0.188      6.056      0.000       0.767       1.505
Antigüedad              -0.2694      0.038     -7.003      0.000      -0.345      -0.194
Latitud                226.8816     44.174      5.136      0.000     140.044     313.719
Fecha_transaccion        5.1348      1.555      3.303      0.001       2.079       8.191
==============================================================================
Omnibus:                      232.810   Durbin-Watson:                   2.154
Prob(Omnibus):                  0.000   Jarque-Bera (JB):             3644.713
Skew:                           2.038   Prob(JB):                         0.00
Kurtosis:                      16.953   Cond. No.                     1.77e+07
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 1.77e+07. This might indicate that there
```

```
are
strong multicollinearity or other numerical problems.
```

In [14]:
```python
# Evaluación del mejor modelo en test
X_train_sel = X_train[variables_seleccionadas]
X_test_sel = X_test[variables_seleccionadas]

modelo_sel = LinearRegression()
modelo_sel.fit(X_train_sel, y_train)

y_pred_sel = modelo_sel.predict(X_test_sel)

MAE_sel = metrics.mean_absolute_error(y_test, y_pred_sel)
MSE_sel = metrics.mean_squared_error(y_test, y_pred_sel)
RMSE_sel = np.sqrt(MSE_sel)
R2_sel = metrics.r2_score(y_test, y_pred_sel)

print("=== COMPARACIÓN DE RENDIMIENTO ===\n")
print("Modelo Completo vs Modelo Seleccionado:")

print(f"\nModelo Completo (todas las variables):")
print(f"  MAE: {MAE:.4f}")
print(f"  RMSE: {RMSE:.4f}")
print(f"  R²: {R2:.4f}")

print(f"\nModelo Seleccionado ({variables_seleccionadas}):")
print(f"  MAE: {MAE_sel:.4f}")
print(f"  RMSE: {RMSE_sel:.4f}")
print(f"  R²: {R2_sel:.4f}")
```

```
=== COMPARACIÓN DE RENDIMIENTO ===

Modelo Completo vs Modelo Seleccionado:

Modelo Completo (todas las variables):
  MAE: 6.1848
  RMSE: 8.5772
  R²: 0.5601

Modelo Seleccionado (['Distancia_estacion', 'Tiendas_conveniencia', 'Antigüe
dad', 'Latitud', 'Fecha_transaccion']):
  MAE: 6.1424
  RMSE: 8.5266
  R²: 0.5652
```

In [15]:
```python
# Visualización del modelo seleccionado
plt.figure(figsize=(10, 5))

plt.subplot(1, 2, 1)
plt.scatter(y_test, y_pred, alpha=0.6, label='Modelo Completo')
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--',
plt.xlabel('Valor Real')
plt.ylabel('Valor Predicho')
plt.title('Modelo Completo')
plt.legend()
```

```
plt.subplot(1, 2, 2)
plt.scatter(y_test, y_pred_sel, alpha=0.6, label='Modelo Seleccionado')
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--',
plt.xlabel('Valor Real')
plt.ylabel('Valor Predicho')
plt.title(f'Modelo Seleccionado: {variables_seleccionadas}')
plt.legend()

plt.tight_layout()
plt.show()
```

